

Onboarding tecnico

Plataforma DTI Nijar

Documento interno del equipo IT DIGITTAL pensado para que cualquier persona que se incorpore al proyecto pueda entender que estamos construyendo, por que, y como trabajar sobre el codigo sin ir a ciegas.

Expediente 18962/2025 · **Adjudicatario** IT DIGITTAL · **Cliente** Ayuntamiento de Nijar

Confidencial · Uso interno IT DIGITTAL

Version 1.0

1. Bienvenido/a al equipo

Si estás leyendo esto es porque te has incorporado al equipo de la **Plataforma DTI Níjar** o vas a tocar el código por primera vez. Bienvenido/a.

Este proyecto tiene varias particularidades que conviene tener claras antes de abrir el editor:

- Es un proyecto **público financiado con fondos NextGenerationEU**, lo que implica obligaciones formales de comunicación, cumplimiento DNSH, antifraude y conservación documental que no son habituales en otros proyectos.
- Es un proyecto con **48 meses de mantenimiento posterior** a la entrega. Lo que tú escribas hoy tiene que ser mantenible por otra persona en 2030.
- Es un proyecto con **cumplimiento ENS Nivel Medio + RGPD + WCAG 2.1 AA obligatorios**. No son negociables.
- Es un proyecto **no genérico**: aunque construimos software, lo construimos para un destino turístico concreto (Níjar, Almería), con sus especificidades culturales, lingüísticas y de territorio.

Ninguna de estas particularidades te va a complicar el día a día si entiendes desde el principio cómo encajan. Este documento intenta justo eso.

Si después de leerlo te queda algo confuso, **pregunta**. Las preguntas inteligentes al principio del proyecto valen oro y son siempre mejor opción que asumir cosas y descubrirlas tres meses tarde en code review.

2. El proyecto en 5 minutos

Quién contrata y quién ejecuta

- **Cliente**: Ayuntamiento de Níjar (Almería). Es un municipio costero del este andaluz, con ~31.000 habitantes y un territorio dominado por el Parque Natural de Cabo de Gata. Tiene picos turísticos importantes en verano y Semana Santa.
- **Adjudicatario**: IT DIGITTAL (nosotros).
- **Expediente**: 18962/2025.
- **Marco**: Plan de Recuperación, Transformación y Resiliencia — Componente 14 «Plan España Digital» — financiación NextGenerationEU al 100%.

Cuánto y cuándo

- **Importe**: 173.906,60 € con IVA.
- **Plazo de implantación inicial**: 8 semanas desde el inicio del contrato.
- **Plazo de mantenimiento posterior (C.1)**: 48 meses tras el SAT.

Qué se entrega

El contrato se divide en cuatro actuaciones que internamente llamamos por su código del Pliego:

- **A.1 — Tótems digitales**: 2 tótems físicos exteriores en Rodalquilar y Los Albaricoques, con información turística en 4 idiomas, accesibles 24/7.
- **A.2 — Smart Office**: 9 sensores ambientales en dependencias municipales que reportan en tiempo real (CO₂, temperatura, humedad, ruido, aforo, meteorología).
- **A.3 — Big Data turístico**: escucha de redes sociales, analítica de la web turística, análisis de sentimiento

del destino.

- **B.2 — Plataforma + Chatbot IA:** backend que orquesta todo lo anterior, CMS multicanal y chatbot conversacional 24/7 multilingüe.

A esto se añaden **requisitos transversales** que afectan a todas las actuaciones: cumplimiento ENS Nivel Medio, RGPD, WCAG 2.1 AA, FIWARE, UNE 178104, DNSH, visibilidad PRTR.

Hitos de pago

El contrato se factura en 4 hitos:

- **H1** — Plan Director y arquitectura aprobada (semana 2).
- **H2** — Desarrollo backend + CMS + chatbot (semana 5).
- **H3** — Despliegue tótems + integración sensores + integración social listening (semana 7).
- **H4** — SAT firmado, plataforma en producción, formación impartida (semana 8).

A partir de H4 arranca el **C.1**: 48 meses de operación, con informes mensuales, soporte 24/7 y SLA de disponibilidad $\geq 99.5\%$.

3. Las 5 piezas que construimos

Pieza 1: Tótems digitales (A.1)

Pantallas exteriores certificadas IP65/IK10 instaladas en dos pedanías con alta afluencia turística. Funcionan 24/7, sin autenticación de usuario.

Qué hacen: muestran información turística (recursos, eventos, mapas, alertas), permiten interactuar con el chatbot y soportan modos de accesibilidad (alto contraste, texto grande, bucle inductivo).

Cómo lo hacen técnicamente: el tótem es un mini-PC industrial con un Linux embebido que abre un navegador en kiosk contra una URL de la plataforma (`/totem`). Esa URL devuelve una SPA HTML5 + JS que hace polling al backend cada 30 segundos para refrescar contenidos. Toda la inteligencia vive en el backend, el tótem es solo el “cristal”.

Por qué es así: si el tótem fuera autónomo, cada cambio de contenido obligaría a desplazar a un técnico físicamente. Con esta arquitectura, los gestores del Ayuntamiento publican un cambio en el CMS y aparece en los tótems en menos de 5 minutos.

Pieza 2: Smart Office (A.2)

Red de 9 sensores ambientales instalados en dependencias municipales (vestíbulo, sala de atención al ciudadano, despacho dirección, cubierta del edificio).

Qué hacen: miden CO₂, temperatura, humedad, ruido, aforo y datos meteorológicos cada 60 segundos.

Cómo lo hacen técnicamente: cada sensor publica vía MQTT en el broker (`nijar/sensors/{tipo}/{ubicacion}`). Un servicio worker (`mqtt-subscriber`) está suscrito al broker, valida el payload, lo enriquece con timestamp y lo persiste en PostgreSQL. Las alertas (CO₂ > 1500 ppm, ruido > 80 dB, etc.) las dispara Prometheus contra los datos persistentes.

Por qué es así: MQTT es el estándar de facto en IoT industrial por su bajo consumo y su modelo publish/subscribe. PostgreSQL es suficiente para nuestro volumen (9 sensores × 60s = ~13.000 observaciones/día), no necesitamos InfluxDB ni TimescaleDB todavía.

Pieza 3: Big Data turístico (A.3)

Conjunto de pipelines que recolectan información de fuentes externas y la convierten en cuadros de mando para el Ayuntamiento.

Qué hace: - Conector de **X (Twitter)**: busca menciones de Níjar, Cabo de Gata, Mónsul, etc. - Conector de **Facebook**: procesa publicaciones de páginas oficiales. - Conector de **Instagram**: rastrea hashtags relevantes. - Conector de **Google Analytics 4**: importa métricas de la web turística. - **Pipeline NLP propio**: clasifica cada mención en sentimiento (positivo/neutro/negativo) y categoría (queja/petición/elogio) en los 4 idiomas del proyecto.

Cómo lo hace técnicamente: hay un worker (`social-worker`) que se ejecuta cada 15 minutos. Por configuración, todos los conectores soportan **dry-run** (modo sintético sin llamar a APIs externas), lo que es crítico para desarrollo local sin necesidad de tokens reales.

Por qué es así: separar conector e ingesta del análisis nos permite cambiar fuentes sin tocar el pipeline NLP. El dry-run es una decisión consciente para que el equipo pueda trabajar sin gestionar credenciales de redes sociales.

Pieza 4: Plataforma core + CMS (B.2)

Es la pieza central. Backend FastAPI que expone una API REST documentada en OpenAPI 3.1 con 30 endpoints, una BBDD PostgreSQL con 22 tablas, un CMS multicanal y los frontales (web turística, dashboard administrativo, plantilla del tótem, app).

Qué hace: gestiona usuarios autenticados, recursos turísticos, eventos, alertas, banners, FAQs del chatbot, observaciones IoT, menciones sociales, KPIs, logs de auditoría. Es el “cerebro” del sistema.

Cómo lo hace técnicamente: arquitectura limpia en capas (router → service → repository → modelo ORM). Validación con Pydantic v2 en frontera. Auth con OAuth2 + JWT. RBAC con 5 roles. Auditoría automática mediante mixin SQLAlchemy.

Pieza 5: Chatbot IA (B.2)

Asistente conversacional disponible 24/7 en 4 idiomas a través del tótem, la web y la app.

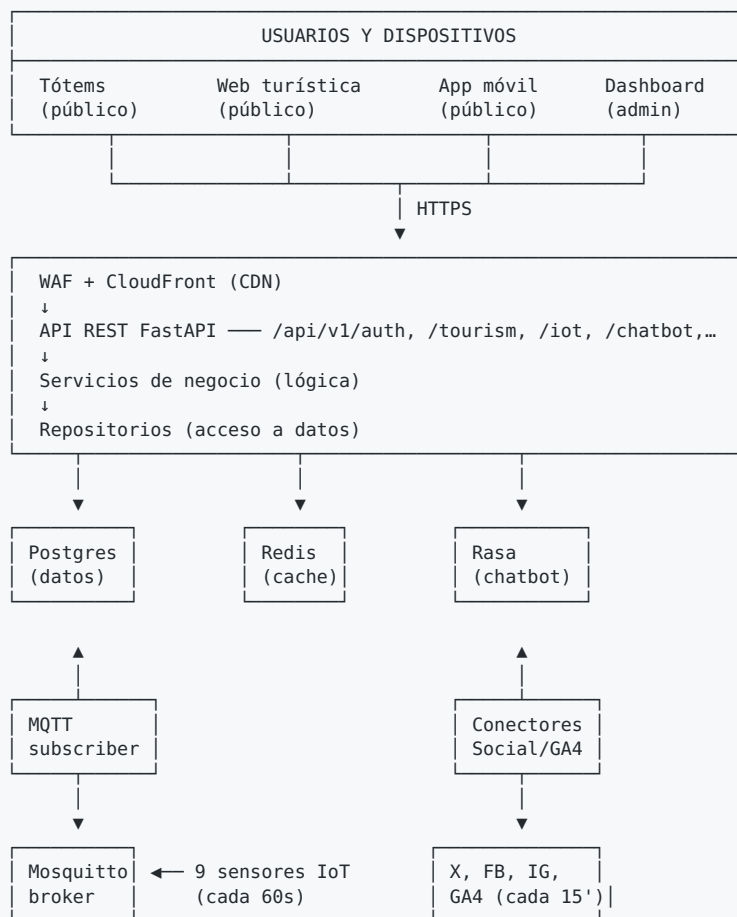
Qué hace: responde a las 22 FAQs base sobre el destino (cómo llegar, horarios del Parque, playas accesibles, alquiler de coches, restaurantes, eventos), detecta cuando una pregunta queda fuera de su dominio y deriva a contacto humano cuando se le pide.

Cómo lo hace técnicamente: hay dos motores intercambiables: - **Motor lexical** (por defecto en local): matching de keywords contra el corpus de FAQs. Sin GPU, sin entrenamiento. Útil para desarrollo y como fallback en producción. - **Motor Rasa** (Rasa Open Source 3.6): NLU completo con clasificación de intents y reconocimiento de entidades. Requiere entrenamiento (3-5 minutos en el primer arranque).

Por qué es así: Rasa Open Source nos garantiza no depender de modelos propietarios (OpenAI, Anthropic) ni pagar tokens. El doble motor permite que cualquiera del equipo trabaje sobre el chatbot sin tener que entrenar Rasa cada vez.

4. Arquitectura mental

Si tuvieras que explicarle a alguien externo cómo funciona la plataforma en una pizarra, este sería el dibujo:



Cómo viaja un dato típico

Caso 1: un visitante consulta los horarios del Parque en el tótem.

1. El tótem (que está en `/totem` cargado en kiosco) hace `GET /api/v1/tourism/recursos?categoria=parque-natural`.
2. La API consulta cache Redis. Si está, lo devuelve. Si no, consulta PostgreSQL y cachea.
3. El tótem renderiza la lista de recursos en el idioma seleccionado.
4. Toda la operación se registra en logs estructurados (Loki en producción, stdout en local).

Caso 2: un sensor de CO₂ reporta una observación.

1. El sensor publica en `nijar/sensors/co2/sala-reuniones` un payload JSON.
2. El broker Mosquitto recibe y entrega al subscriber MQTT.
3. El servicio `mqtt-subscriber` valida el schema con Pydantic, enriquece con timestamp UTC y lo persiste como `Observacion` en PostgreSQL.
4. Prometheus scrapea las métricas de la API cada 15 segundos. Si el último valor de CO₂ supera el umbral durante más de 5 minutos, dispara la alerta.
5. Alertmanager envía notificación al SOC.

Caso 3: un usuario pregunta al chatbot “What time does Cabo de Gata open?”

1. El frontal del chatbot hace `POST /api/v1/chatbot/message` con el texto y el idioma.
2. La API delega al adapter (`chatbot_rasa_adapter` si Rasa está activo, `chatbot_service` si lexical).
3. Rasa clasifica el intent como `consultar_horario_parque` con confidence 0.92.
4. Como la confidence supera el umbral, el adapter consulta la respuesta correspondiente del corpus en el idioma EN.
5. La interacción se registra en la tabla `interaccion_chatbot` para análisis posterior.

6. Se devuelve la respuesta al frontal.

5. El stack técnico

Capa	Tecnología	Versión
Backend	Python	3.12
Framework HTTP	FastAPI	0.111
Validación	Pydantic	v2
ORM	SQLAlchemy	2.0 (async)
Migraciones	Alembic	1.13
Driver BBDD	asyncpg	0.29
Base de datos	PostgreSQL + PostGIS	16
Cache	Redis	7
MQTT broker	Eclipse Mosquitto	2.x
Cliente MQTT	paho-mqtt	1.6
Chatbot	Rasa Open Source	3.6
NLP propio	spaCy + transformers	3.7 / 4.40
Frontend SPA	React + Vite + Tailwind	18 / 5 / 3
Auth	OAuth2 password flow + JWT	—
Hashing	bcrypt vía passlib	—
Tests	pytest + httpx	—
Containers	Docker	24+
Orquestación local	Docker Compose	v2
Orquestación prod	Kubernetes (EKS)	1.30
IaC	Terraform	1.7
CI/CD	GitHub Actions	—
Observabilidad	Prometheus + Grafana + Loki	—
Cloud	AWS (eu-central-1, Frankfurt)	—
WAF	AWS WAF v2 + AWSManagedRules	—
CDN	CloudFront	—
Cifrado	KMS con rotación	—

Filosofía: stack abierto, sin servicios propietarios cerrados (ni AppSync, ni Lambda específico, ni RDS Aurora). Si mañana el Ayuntamiento decide cambiar de proveedor cloud, debe poder hacerse en semanas, no en años.

6. Estructura del repositorio

Cuando abras la carpeta del proyecto vas a ver esto:

nijar-dti-platform/	
└─ src/nijar_dti/	← TODO el código Python del backend
└─ │ api/v1/	← Routers REST (FastAPI). Aquí tocas para cambiar endpoints.
└─ │ core/	← Configuración, settings (Pydantic), seguridad, JWT.
└─ │ data/	← Datos seed (recursos turísticos, FAQs, etc.).
└─ │ ga4/	← Conector Google Analytics 4.
└─ │ mqtt/	← Subscriber MQTT y parser de payloads.
└─ │ nlp/	← Pipeline NLP propio (sentimiento, categorías).
└─ │ repositories/	← Acceso a datos (CRUD sobre modelos ORM).
└─ │ schemas/	← Schemas Pydantic v2 (entrada y salida de la API).
└─ │ services/	← Lógica de negocio. Aquí va el "cómo".
└─ │ social/	← Conectores X, Facebook, Instagram.
└─ │ models.py	← Modelos SQLAlchemy.
└─ alembic/	
└─ │ versions/	← Migraciones de BBDD. Una por cambio de schema.
└─ frontend/	← SPAs y plantillas (TypeScript + Vite).
└─ │ dashboard/	← Panel admin del Ayuntamiento.
└─ │ totem/	← Plantilla que carga el tótem en kiosco.
└─ rasa/	← Configuración Rasa Open Source.
└─ │ domain.yml	← Intents, respuestas, slots, entidades.
└─ │ data/nlu.yml	← Ejemplos de entrenamiento.
└─ │ data/rules.yml	← Reglas conversacionales.
└─ │ data/stories.yml	← Historias para el modelo.
└─ │ config.yml	← Pipeline NLU + políticas.
└─ infra/	← Infraestructura como código.
└─ │ terraform/	← AWS (EKS, RDS, S3, WAF, KMS, CloudFront, etc.).
└─ │ k8s/	← Manifiestos Kubernetes (deployments, services, etc.).
└─ │ observability/	← Dashboards Grafana + alertas Prometheus.
└─ docs/	← Documentación técnica (markdown).
└─ │ adr/	← Architectural Decision Records (21 ADRs).
└─ │ architecture/	← Diagramas y arquitectura.
└─ │ data-model/	← Schemas JSON FIWARE NGSI-LD.
└─ │ operations/	← Runbooks operativos para C.I.
└─ │ security/	← Plan pentest, política seguridad.
└─ tests/	← pytest. Suite de 202 tests.
└─ │ unit/	← Tests unitarios.
└─ │ integration/	← Tests integración API.
└─ │ coherence/	← Tests coherencia documental.
└─ scripts/	← Scripts auxiliares de desarrollo.
└─ │ dev_up.sh	← Arranque local en Linux/macOS.
└─ windows/	← Scripts de despliegue local Windows.
└─ │ setup.bat / .ps1	← Primer arranque.
└─ │ start.bat / .ps1	← Arrancar.
└─ │ stop.bat / .ps1	← Parar.
└─ │ status.bat / .ps1	← Ver estado.
└─ │ logs.bat / .ps1	← Ver logs.
└─ │ reset.bat / .ps1	← Borrar y empezar de cero.
└─ .github/workflows/	← CI/CD (build, security, deploy).
└─ docker-compose.yml	← Orquestación local.
└─ Dockerfile	← Imagen del backend.
└─ pyproject.toml	← Dependencias Python (poetry).
└─ alembic.ini	← Configuración Alembic.
└─ .env.example	← Plantilla de variables de entorno.
└─ README.md	← README principal del proyecto.
└─ README-Windows.md	← Guía completa de despliegue Windows.
└─ LEEME-PRIMERO.txt	← Puerta de entrada para nuevos compañeros.

“¿Dónde toco para...?”

Quiero...	Toco en...
Añadir un endpoint REST nuevo	<code>src/nijar_dti/api/v1/</code> (router) + <code>services/</code> (lógica) + <code>schemas/</code> (modelos Pydantic)
Cambiar la lógica de un endpoint existente	El <code>service</code> correspondiente
Añadir una tabla/columna en BBDD	<code>models.py</code> + crear migración con <code>alembic revision --autogenerate</code>
Cambiar respuestas del chatbot lexical	<code>src/nijar_dti/data/seeds/faqs.py</code>
Cambiar respuestas del chatbot Rasa	<code>rasa/domain.yml</code> + <code>rasa/data/nlu.yml</code>
Añadir un sensor IoT nuevo	<code>src/nijar_dti/mqtt/parser.py</code> (validación payload) + seed
Tocar el dashboard admin	<code>frontend/dashboard/</code>
Tocar la plantilla del tótem	<code>frontend/totem/</code>
Cambiar config Terraform AWS	<code>infra/terraform/</code>
Cambiar manifiestos K8s	<code>infra/k8s/</code>
Añadir un dashboard Grafana	<code>infra/observability/grafana-dashboards/</code>
Añadir una alerta Prometheus	<code>infra/observability/prometheus-alerts.yaml</code>
Añadir un test	<code>tests/unit/</code> o <code>tests/integration/</code> según aplique

7. Decisiones técnicas que no se tocan a la ligera

Son decisiones que están consolidadas en ADRs (`docs/adr/`) y firmadas en la Memoria Técnica. Cambiarlas requiere acuerdo del Comité de Cambios y, en algunos casos, del Ayuntamiento.

7.1. AWS región eu-central-1 (Frankfurt)

Por qué: el RGPD obliga a que los datos personales de ciudadanos europeos se traten dentro del Espacio Económico Europeo. eu-central-1 (Frankfurt) está dentro de la UE. La región tiene SOC 2, ISO 27001 e ISO 27018 acreditadas.

Implicación: nada de servicios que solo estén en us-east-1. Nada de modelos de IA propietarios alojados fuera de la UE (esto descarta OpenAI directamente; Bedrock con modelos europeos sería negociable, pero no estaba justificado).

7.2. Rasa Open Source y no un LLM propietario

Por qué: el Pliego prohíbe expresamente la dependencia de modelos propietarios. Además, no podemos enviar consultas de ciudadanos a OpenAI/Anthropic sin un análisis previo de transferencias internacionales y un DPA específico que el Ayuntamiento no quiere firmar.

Implicación: si te tienta integrar GPT-4 o Claude por la mejor calidad de respuestas, **no lo hagas** sin antes pasarlo por el Comité. La calidad del chatbot la mejoramos con más FAQs en `rasa/data/nlu.yml` , no añadiendo una dependencia externa.

7.3. FIWARE Smart Data Models + NGSI-LD

Por qué: el Pliego (PPT pág. 42) lo exige explícitamente. Es el estándar europeo de interoperabilidad para Smart Cities, promovido por la Comisión Europea.

Implicación: nuestros modelos `RecursoTuristico` , `EventoTuristico` , `Observacion` , etc., usan URN del tipo `urn:ngsi-ld:RecursoTuristico:nijar:001` . No los conviertas a IDs autoincrementales aunque sean más cómodos

de leer.

7.4. ENS Nivel Medio en las 5 dimensiones

Por qué: la Plataforma trata datos del personal del Ayuntamiento (administradores autenticados) y de gestión municipal. El RD 311/2022 exige Nivel Medio.

Implicación práctica para tu día a día: - **MFA obligatorio para todos los administradores.** No hay excepciones, ni “es que estoy en local”. - **Cifrado en reposo y en tránsito.** KMS en AWS, TLS 1.2+ extremo a extremo. - **Logs de auditoría inmutables** durante 12 meses para acciones que afectan datos. - **Pentest anual obligatorio** por tercero certificado. Tu código va a ser auditado por gente que no lo escribió. - **Borrado seguro NIST 800-88** para soportes que se retiran.

7.5. WCAG 2.1 AA en todos los frontales

Por qué: el RD 1112/2018 lo exige para todos los sitios y aplicaciones del sector público. No es opcional.

Implicación práctica: - **axe-core es bloqueante en CI.** Si introduces una violación de accesibilidad, no se hace merge. - **Lighthouse mínimo 90/100** en todas las páginas. - Si tienes dudas sobre una decisión de UI (un color, un tamaño de fuente, una interacción), valida con axe-core en local antes de hacer push.

7.6. Principio DNSH (Do No Significant Harm)

Por qué: el PRTR financia el proyecto bajo la condición de no causar perjuicio significativo a 6 objetivos medioambientales europeos.

Implicación práctica: nada de criptominería oculta (cosa que nadie querría hacer pero que conviene mencionar), uso responsable de recursos cloud (instancias spot cuando sea posible, autoescalado, scheduled shutdown de entornos no productivos). Hardware con vida útil mínima de 7 años. Reciclaje conforme al RD 110/2015.

7.7. Visibilidad PRTR/NextGenerationEU

Por qué: el art. 34 del Reglamento (UE) 2021/241 obliga a hacer visible la financiación europea en todos los soportes públicos del proyecto.

Implicación práctica: el footer del frontal del tótem, de la web turística y del dashboard administrativo lleva los logos exigidos (UE + bandera, NextGenerationEU, PRTR, Componente 14, Ayuntamiento de Níjar). Esto no se quita “para que quede más limpio el diseño”. Es contractual.

8. Cómo desplegar en local

La guía completa está en `README-Windows.md` (extracción del ZIP unificado). Aquí solo el resumen mental de lo que pasa para que sepas qué esperas:

Requisitos en tu máquina

- Windows 10/11 con WSL2 habilitado.
- Docker Desktop funcionando.
- ~10 GB libres en disco.
- Idealmente 16 GB de RAM (con 8 GB sólo perfil mínimo, sin Rasa).

Lo que NO necesitas

- Python instalado en Windows.
- Node.js instalado en Windows.

- PostgreSQL local.
- Cuenta AWS.
- Tokens de Twitter, Facebook o Instagram (los conectores arrancan en dry-run por defecto).
- Laragon, XAMPP, WAMP. **Nada de eso aporta**, todo va por Docker.

Pasos

1. Mover el ZIP a `C:\dev\` y extraer.
2. Doble clic en `windows\setup.bat` (sólo la primera vez). Tarda 2-10 min.
3. Doble clic en `windows\start.bat` para arrancar el perfil mínimo.
4. Esperar a que se abra el navegador en `http://localhost:8000/docs`.
5. Probar login con `admin@nihar.es` / `CambiarEnPrimerArranque#2026`.

Perfiles disponibles

- **Mínimo** (`start.ps1`): API + Postgres + Redis + MQTT broker. Suficiente para trabajar sobre la API y el CMS. ~1.5 GB RAM.
- **Workers** (`start.ps1 -Workers`): además, MQTT subscriber + Social Worker. Necesario si vas a tocar el flujo IoT o el pipeline de redes sociales. ~2.5 GB RAM.
- **Completo** (`start.ps1 -Workers -Rasa`): incluye Rasa entrenado. Necesario si vas a tocar el chatbot avanzado. ~5 GB RAM.

Si algo falla

- Los logs hablan claro: `windows\logs.bat` para la API, `windows\logs.ps1 -Service db` para Postgres, etc.
- `windows\status.bat` te dice qué contenedor está OK y cuál no.
- Si la BBDD se queda en estado raro, `windows\reset.bat` lo borra todo y vuelve a empezar.
- Sección 7 del `README-Windows.md` cubre los 11 problemas más frecuentes.

9. Tu primera tarea típica

Para que veas el flujo de extremo a extremo, supón que te asignan esta tarea típica:

Ticket #1234: "Añadir un nuevo recurso turístico (categoría 'restaurantes') con su descripción en los 4 idiomas y mostrarlo en los tótems."

Pasos mentales

1. Entender el modelo de datos.

El recurso turístico ya existe, lo defines en `src/nihar_dti/data/seeds/recursos_turisticos.py` o lo creas vía el endpoint `POST /api/v1/tourism/recursos`. Los modelos están en `models.py`, los schemas Pydantic en `schemas/tourism.py`.

2. Decidir el camino: seed o API.

- Si es un dato fijo del proyecto (ej: la oficina de turismo principal), va en seed para que se cargue en cada despliegue.
- Si es un dato que el Ayuntamiento gestiona (ej: un restaurante que cambia con la temporada), va vía CMS por el endpoint de la API.

3. Si es seed, lo añades en `recursos_turisticos.py` siguiendo el patrón existente: URN NGSI-LD, coordenadas GPS reales, descripciones en 4 idiomas, categoría, accesibilidad, etc.

4. Lanzar test local.

```
docker compose exec api pytest tests/unit/test_recursos_turisticos.py
```

5. Reset para que se cargue el seed nuevo.

```
.\windows\reset.bat # confirma con SI
.\windows\start.bat
```

6. Verificar en local.

- Ir a `http://localhost:8000/docs`, login, llamar a `GET /api/v1/tourism/recursos?categoria=restaurantes`.
- Ir a `http://localhost:8000/totem` y ver que aparece.

7. Commit y PR.

```
git checkout -b feature/1234-restaurant-tabula-rasa
git add src/nijar_dti/data/seeds/recursos_turisticos.py
git commit -m "feat(tourism): añadir restaurante Tabula Rasa al seed (#1234)"
git push origin feature/1234-restaurant-tabula-rasa
```

Abrir el PR. **No hagas merge tú mismo**. La review la hace otra persona.

8. Esperar al CI.

GitHub Actions va a: - Lanzar la suite de tests (202 tests). - Lanzar pip-audit, OSV-Scanner y Trivy contra dependencias e imágenes. - Lanzar axe-core sobre los frontales. - Lanzar Lighthouse. - Validar que las migraciones aplican y revierten.

Si algún check falla, el merge está bloqueado. Si todo OK + reviewer aprueba, **merge a main** y el deploy se ejecuta automáticamente.

10. Workflow del equipo

Git

- **Rama protegida:** `main`. No se pushea directo, solo merges desde PR.
- **Convención de ramas:** `feature/NNNN-descripcion-corta`, `fix/NNNN-descripcion`, `chore/...`.
- **Convención de commits:** Conventional Commits. `feat(scope): mensaje`, `fix(scope): mensaje`, `chore: mensaje`. El scope suele ser el módulo (`tourism`, `iot`, `chatbot`, etc.).
- **PR:** descripción clara, ticket asociado, capturas si afecta a UI, lista de tests añadidos.

Code review

- Mínimo **una persona** debe aprobar el PR.
- Cambios de seguridad (RBAC, auth, cifrado): revisión adicional del especialista de ciberseguridad.
- Cambios en la BBDD (modelos o migraciones): revisión adicional del arquitecto.

CI/CD

Tres workflows en `.github/workflows/`:

- `build.yml`: compila imágenes Docker, lanza tests, valida coherencia.
- `security.yml`: pip-audit + OSV-Scanner + Trivy + bandit. Bloqueante si CVE crítico.
- `deploy.yml`: despliega en EKS (staging → prod) con aprobación manual para prod.

ADRs

Cuando tomes una decisión técnica con impacto a futuro, **escribe un ADR**. Hay una plantilla en `docs/adr/template.md`. El ADR se versiona junto al código.

Reuniones

- **Daily**: 15 minutos diarios, 9:30 CET.
- **Refinement**: lunes por la tarde, revisión del backlog.
- **Demo + retro**: cada 2 viernes.
- **Comité de Cambios** (con el Ayuntamiento): mensual.

11. Glosario de términos del Pliego

Términos que vas a leer en commits, en tickets, en mensajes del Ayuntamiento. Mejor saber qué significan desde el primer día.

Término	Significado
DTI	Destino Turístico Inteligente. Modelo SEGITTUR para destinos que aplican tecnología, sostenibilidad e innovación.
PRTR	Plan de Recuperación, Transformación y Resiliencia (España). Marco que financia el proyecto.
NextGenerationEU	Programa europeo de ~750.000 M€ post-COVID. Origen último de los fondos.
C14	Componente 14 del PRTR — “Plan España Digital”. Es el componente concreto del que viene la financiación.
DNSH	Do No Significant Harm. Principio de no causar perjuicio significativo a 6 objetivos medioambientales.
DACI	Declaración de Ausencia de Conflicto de Intereses. Obligatoria para empleados clave del proyecto.
MRR	Mecanismo de Recuperación y Resiliencia. Reglamento (UE) 2021/241 que regula NextGenerationEU.
A.1, A.2, A.3, B.2	Códigos de las actuaciones del Pliego (tótems, Smart Office, Big Data, Plataforma+Chatbot).
C.1	Mantenimiento posterior a la entrega. 48 meses de operación tras el SAT.
SAT	Site Acceptance Test. Prueba de aceptación en el sitio que firma el Ayuntamiento al fin de la implantación.
FAT	Factory Acceptance Test. Pruebas en pre-producción antes del despliegue.
UAT	User Acceptance Test. Pruebas que ejecuta personal del Ayuntamiento en su día a día.
ENS	Esquema Nacional de Seguridad. RD 311/2022. Niveles: Básico, Medio, Alto.
RGPD	Reglamento General de Protección de Datos. UE 2016/679.
LOPDGDD	Ley Orgánica 3/2018 de Protección de Datos. Versión española del RGPD.
DPO	Delegado de Protección de Datos. Persona del Ayuntamiento responsable RGPD.
DPA	Data Processing Agreement. Acuerdo art. 28 RGPD entre

	Responsable y Encargado.
DPIA	Data Protection Impact Assessment. Evaluación de impacto en protección de datos (art. 35 RGPD).
WCAG 2.1 AA	Web Content Accessibility Guidelines, nivel doble A. Estándar de accesibilidad web.
FIWARE	Iniciativa europea de estándares de interoperabilidad para Smart Cities.
NGSI-LD	Next Generation Service Interfaces Linked Data. Protocolo FIWARE para representar datos.
UNE 178104	Norma española sobre arquitectura de Plataformas DTI. La cumplimos por contrato.
MAGERIT v3	Metodología de análisis de riesgos del CCN-CERT. Lo aplicamos para el análisis ENS.
CCN-STIC	Guías del Centro Criptológico Nacional para implementación ENS.
AEPD	Agencia Española de Protección de Datos. Autoridad nacional RGPD.
OLAF	Oficina Europea de Lucha contra el Fraude. Puede investigar irregularidades en fondos UE.
Tótem A.1-01 / A.1-02	Forma de referirnos a los 2 tótems físicos: Rodalquilar y Los Albaricoques respectivamente.
Smart Office	Las dependencias municipales sensorizadas. NO confundir con coworking.
MFA	Multi-Factor Authentication. Obligatorio para administradores.
RBAC	Role-Based Access Control. Tenemos 5 roles: administrador_tic, gestor_contenidos, analista_datos, operador_smart_office, auditor.
MQTT	Protocolo IoT publish/subscribe. Lo usan los 9 sensores.
KMS	Key Management Service de AWS. Cifrado en reposo.
WAF	Web Application Firewall. Protección capa 7 de la API.

12. Recursos y a quién preguntar

Documentación interna del proyecto

- **README.md** — README principal del proyecto (técnico, asume conocimiento previo).
- **README-Windows.md** — Despliegue local en Windows con troubleshooting.
- **docs/adr/** — 21 Architectural Decision Records que justifican decisiones.
- **docs/architecture/** — Diagramas y arquitectura de alto nivel.
- **docs/operations/** — Runbooks operativos del C.1.
- **docs/security/** — Plan pentest, política de seguridad.

Documentación administrativa entregada al Ayuntamiento

(Hay una copia en el repositorio del proyecto, en una carpeta separada del expediente.)

- Memoria Técnica Final.
- Política de Seguridad ENS.

- Análisis de Riesgos MAGERIT v3.
- DPA, DPIA.
- Plan de Pruebas FAT/SAT/UAT.
- Plan de Reversión.
- Manual de Administración Técnica.
- Manual de Usuario del CMS.
- Plan de Formación + Evaluaciones.
- Catálogo de KPIs.
- Política de Gestión de Cambios C.1.
- Plan de Comunicación PRTR.
- Inventario de Componentes.
- Matriz de Cumplimiento + Trazabilidad.

Documentación externa relevante

- **Pliego de Cláusulas Administrativas Particulares (PCAP)** — el contrato.
- **Pliego de Prescripciones Técnicas (PPT)** — qué construimos.
- **RD 311/2022** — ENS.
- **Reglamento (UE) 2016/679** — RGPD.
- **Orden HFP/1030/2021** — gestión del PRTR.
- **Smart Data Models de FIWARE** — <https://smartdatamodels.org/>

A quién preguntar (cadena de escalado)

Tema	Persona
Cualquier duda no urgente	Compañero/a de equipo más cercano
Decisiones de arquitectura	Arquitecto
Decisiones de seguridad	Especialista Ciberseguridad
Decisiones que afectan al modelo económico	Director del Proyecto
Decisiones de UI/UX	Líder UX
Comunicación con el Ayuntamiento	Director del Proyecto (NO contactar directamente al Ayuntamiento sin pasar por él/ella)
Cuestiones legales o RGPD	Jurídico de IT DIGITAL + DPO del Ayuntamiento

Última cosa

Este documento es vivo. Si encuentras algo confuso, desactualizado o directamente erróneo mientras te incorporas, **abre un PR contra este archivo** y arréglalo para los que vengan detrás. Es la mejor forma de demostrar que ya estás integrado en el equipo.

Bienvenido/a otra vez. A construir.